COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	8/4/26	**DATE SUBMITTED:**	8/4/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

ILO-A

ILOA > main.cpp > main()

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    //create an object
    std::queue<char> myQ;

    myQ.push('V');
    myQ.push('I');
    myQ.push('N');
    myQ.push('C');
    myQ.push('E');

    display(myQ);
    //empty
    std::cout<<"The queue is ";
    if(myQ.empty()){
        std::cout<<"Empty."<< std::endl;
    }else{
        std::cout<<"Not Empty."<<std::endl;
    }

    //size
    std::cout<<"What is the size of queue?: "<< myQ.size()<<std::endl;

    //front
    std::cout<<"The front of the queue is "<< myQ.front()<<std::endl;

    //back
    std::cout<<"The back of the queue is "<< myQ.back()<<std::endl;
```

```
33
34      //pop
35      myQ.pop();
36      display(myQ);
37
38      //push
39      myQ.push('A');
40      display(myQ);
41
42      return 0;
43  }
44  void display(std::queue<char> copyQ){
45      //create a copy of the queue that was passed
46      std::queue<char> c = copyQ;
47      while(!c.empty()){
48          std::cout << " "<< c.front();
49          c.pop();
50      }
51      std::cout <<"\n";
52
53  }
```

Observation:
In this code, we have used the library for queues. The functions were very similar from the past functions that were introduced before this topic. The push is used to add data, pop is used to remove data from the queue, empty is for checking if the queue is empty, front is responsible for printing the front of the queue and lastly back was used to print the end of the queue. We have created a new function to display all of the data inside the queue.

Output:

```
PS C:\Users\TIPQC\Documents\CPE_Urbano\
nin2lr.1vs' '--stdout=Microsoft-MIEngin
terpreter=mi'
 V I N C E
The queue is Not Empty.
What is the size of queue?: 5
The front of the queue is V
The back of the queue is E
 I N C E
 I N C E A
```

ILO B:


Welcome
main.cpp
queueLL.h
queueArrLL.h
ILOB > queueLL.h > displayAll<T>(qNode<T>**, qNode<T>**)
#ifndef QUEUELL_H
#define QUEUELL_H
//create a class the class nod
template <typename T>
class qNode{
public:
T data;
qNode<T> *next;
};
//creates the new node
template <typename T>
qNode<T>* new_node(T newData){
qNode<T>* newNode = new qNode<T>;
newNode->data = newData;
newNode->next = nullptr;
return newNode;
}
//enqueue operation
template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
//create the new node
qNode<T>* newNode = new_node(newData);
//insertint into an empty queue
if((*frontPtr)==nullptr && (*backPtr) == nullptr){
(*frontPtr) = newNode;
(*backPtr) = newNode;
}
// inserting an ite, imto a non empty queue
(*backPtr)->next = newNode;
(*backPtr) = newNode;
}

```cpp

template<typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a temporary node
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);
    //
    //check if the has only 1 node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }


    //delete the node
    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;
}
//check the front
template <typename T>
void frontQ(qNode<T>** frontPtr){
    if((*frontPtr == nullptr)) return;

    qNode<T>* temp = (*frontPtr);
    std::cout<<"The front is "<< temp->data <<std::endl;

}

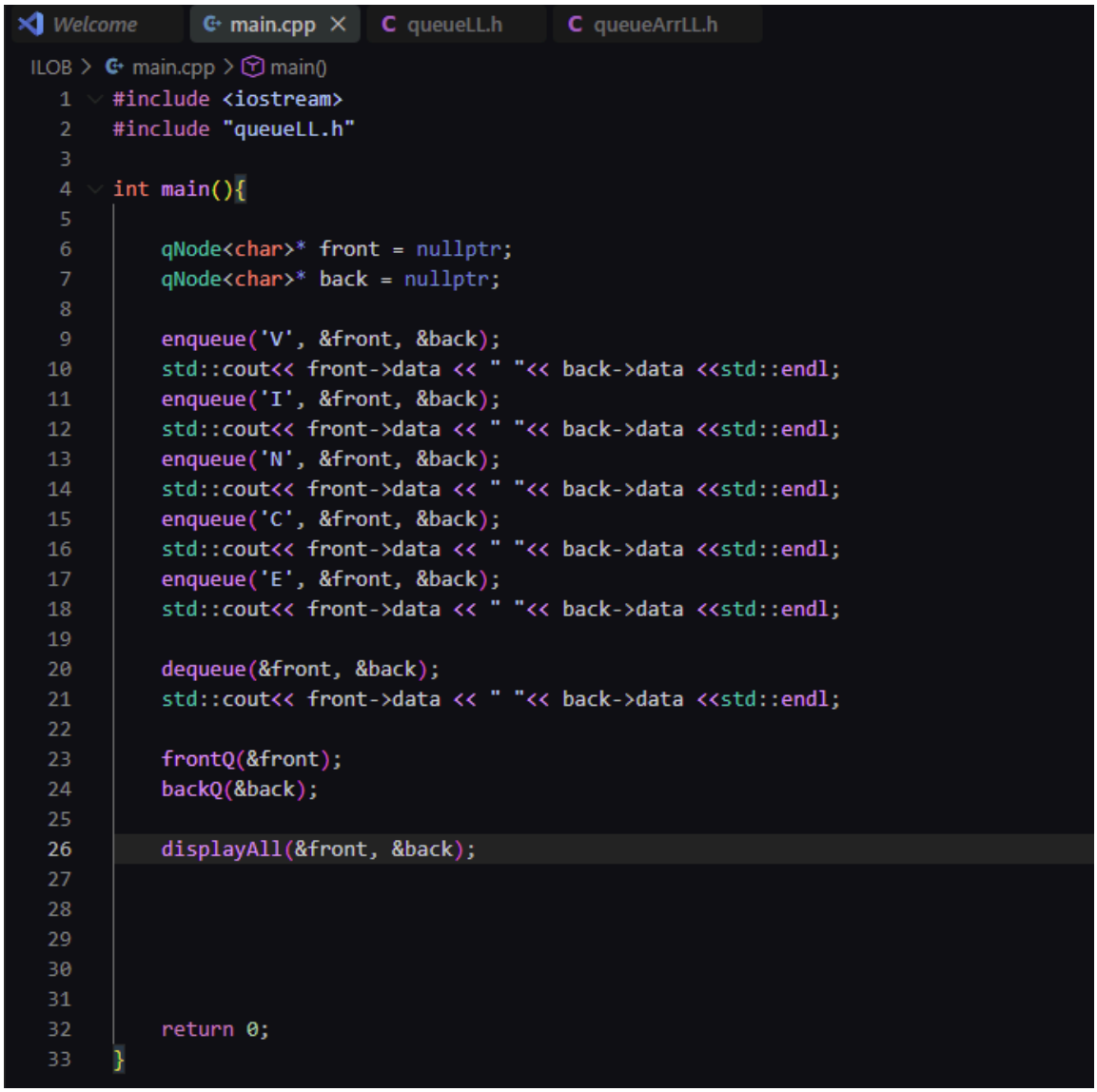
//check the back
template <typename T>
void backQ(qNode<T>** backPtr){
    if((*backPtr == nullptr )) return;

    qNode<T>* temp = (*backPtr);
    std::cout<<"The back is "<< temp->data <<std::endl;

}
```

```
78  //display all
79  template <typename T>
80  void displayAll(qNode<T>** frontPtr, qNode<T>** backPtr){
81      if((*frontPtr) == nullptr && (*backPtr) == nullptr) return;
82
83      qNode<T>* display = (*frontPtr);
84
85      while(display != nullptr){
86
87          std::cout<< display->data <<std::endl;
88          display = display->next;
89      }
90  }
91
92
93  #endif
```

Main:


Welcome
main.cpp
queueLL.h
queueArrLL.h
ILOB > main.cpp > main()
#include <iostream>
#include "queueLL.h"
int main(){
qNode<char>* front = nullptr;
qNode<char>* back = nullptr;
enqueue('V', &front, &back);
std::cout<< front->data << " "<< back->data <<std::endl;
enqueue('I', &front, &back);
std::cout<< front->data << " "<< back->data <<std::endl;
enqueue('N', &front, &back);
std::cout<< front->data << " "<< back->data <<std::endl;
enqueue('C', &front, &back);
std::cout<< front->data << " "<< back->data <<std::endl;
enqueue('E', &front, &back);
std::cout<< front->data << " "<< back->data <<std::endl;
dequeue(&front, &back);
std::cout<< front->data << " "<< back->data <<std::endl;
frontQ(&front);
backQ(&back);
displayAll(&front, &back);
return 0;
}

Output:

```
PS C:\Users\TIPQC\Documents\CPE_Urbano\Queue>
5a0h4q.l3o' '--stdout=Microsoft-MIEngine-Out-u
terpreter=mi'
V N
V C
V E
I E
The front is I
The back is E
I
N
C
E
```

Observation:
During this activity, I have noticed that it is not very different from the past lessons but the main difference of queue to the others is that the first data that we have pushed will be the first one to be removed. The head will always remain as the head of the data even if there is new data that was pushed. While doing the activity I have also noticed that we always have to point an object to the data otherwise it will only print the address of the object that we want to print. It is also very important to create a catcher for the functions because without the checkers, it will return to an error.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected
to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs
to store all the pending jobs somewhere so that it can take them up once its current task is done.
Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.
You may NOT use the STL.
1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

Code:
Printll.h

printll.h × Supplementary.cpp

```
#ifndef PRINTLL_H
#define PRINTLL_H

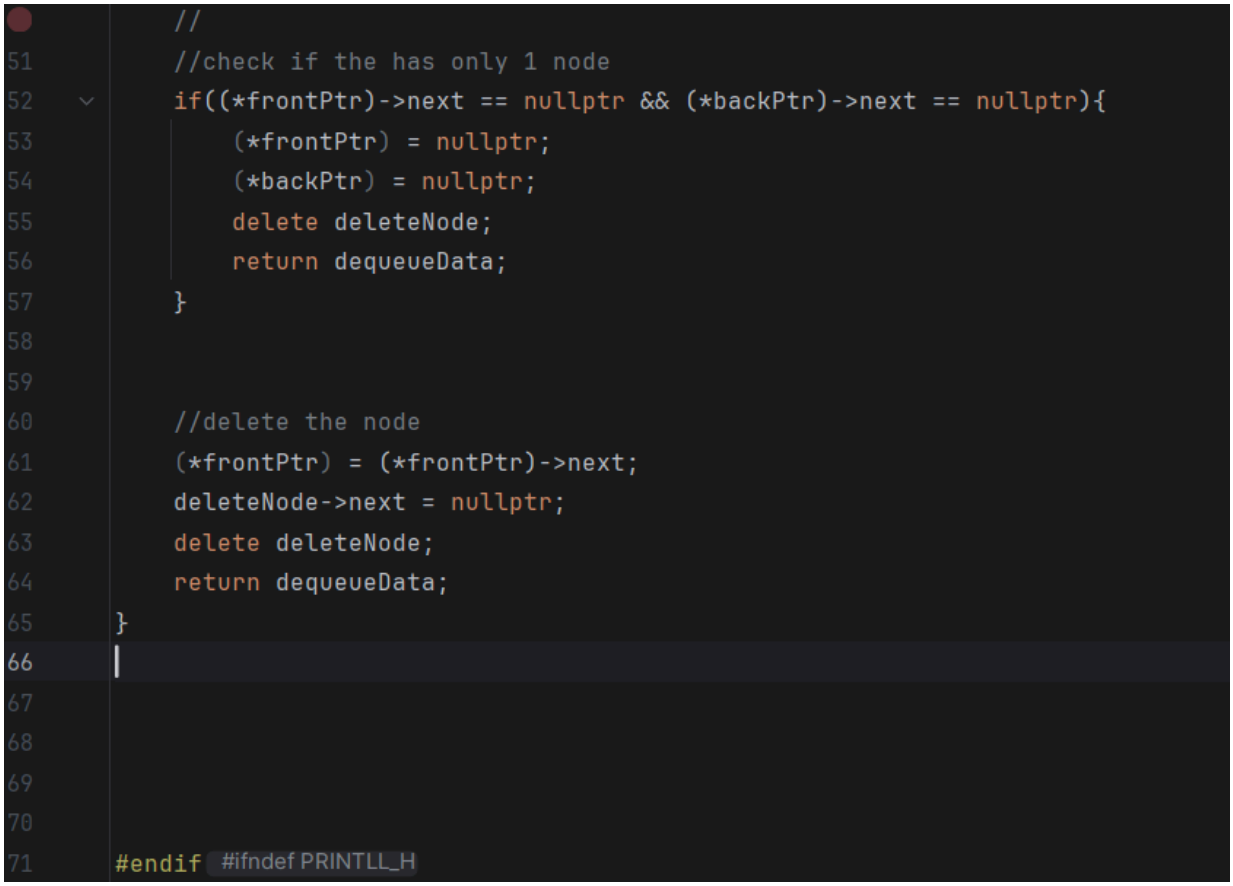
template <typename T>
class qNode{
public:
    T data;
    qNode<T> *next;
};

//creates the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}
//enqueue operation
template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create the new node
    qNode<T>* newNode = new_node(newData);

    //insertint into an empty queue
    if((*frontPtr)==nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;

```

```
30      }
31      // inserting an ite, imto a non empty queue
32      (*backPtr)->next = newNode;
33      (*backPtr) = newNode;
34
35  }
36  template <typename T>
37  bool isEmpty(qNode<T>** frontPtr) {
38      return (*frontPtr == nullptr);
39  }
40
41  template<typename T>
42  T dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
43      if (isEmpty(frontPtr)) {
44          std::cout<<"The queue is empty"<<std::endl;
45          return T();
46      };
47      //create a temporary node
48      qNode<T>* deleteNode = *frontPtr;
49      T dequeueData =deleteNode->data;
```



```
    //
    //check if the has only 1 node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return dequeueData;
    }


    //delete the node
    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;
    return dequeueData;
}


#endif #ifndef PRINTLL_H
```

Main.cpp

```cpp
#include <iostream>
#include "printll.h"
#include <string.h>

class Job {
public:
    int id;
    std::string employeeName;
    int pages;

    Job() {
        id = 0;
        employeeName = "";
        pages = 0;
    }

    void setJob(int i, std::string name, int p) {
        id = i;
        employeeName = name;
        pages = p;
    }
};

class Printer {
private:
    qNode<Job>* front=nullptr;
    qNode<Job>* back = nullptr;

```

```
public:
    void queuePrint(int id, std::string name, int pages) {
    Job newJob;
        newJob.setJob( i: id, name, p: pages);
        enqueue( newJob, &front, &back);
        std::cout<<"in Queue "<< id <<" "<< name << " No. of Pages: " << pages << std::endl;
    }
    void print() {
        if (isEmpty(&front)) {
            std::cout << "Queue is empty." << std::endl;
            return;
        }
        Job currentJob = dequeue(&front, &back);
        std::cout << "Printing: ID: " << currentJob.id << " Name:"<< currentJob.employeeName << " Pages: " << currentJob.pages << std:
    }
};

int main() {
    Printer printer;

    printer.queuePrint( id: 2516918, name: "Vince", pages: 10);
    printer.queuePrint( id: 2516919, name: "Andrew", pages: 5);
    printer.queuePrint( id: 2516920, name: "Urbano", pages: 20);
```

```
    std::cout << std::endl;

    printer.print();
    printer.print();

    std::cout << std::endl;

    printer.queuePrint( id: 2516918, name: "Francisco", pages: 15);

    std::cout << std::endl;

    printer.print();
    printer.print();
    printer.print();

    return 0;
}
```

Output:

```
"C:\Users\Vince Urbano\Documents\Supplementary\HOA5\cmake-build-debug\HOA5
in Queue 2516918 Vince No. of Pages: 10
in Queue 2516919 Andrew No. of Pages: 5
in Queue 2516920 Urbano No. of Pages: 20

Printing: ID: 2516918 Name:Vince Pages: 10
Printing: ID: 2516919 Name:Andrew Pages: 5

in Queue 2516918 Francisco No. of Pages: 15

Printing: ID: 2516920 Name:Urbano Pages: 20
Printing: ID: 2516918 Name:Francisco Pages: 15
Queue is empty.

Process finished with exit code 0
```

Conclusion:

During this activity, I have used a linked list because it allowed me to create a new node which made coding simpler. Inside these nodes, there are different data but it strictly follows the rule of Queue which is first in first out. The class named Job is the class that is responsible for creating the objects. The function setJob is implemented to assign the values to the objects. We also created a Printer class which will be responsible for queueing and dequeueing a value.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Assistance with a bug**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.